Week 7 – Feedforward Neural Networks (MLP) Analysis

Task 1 – Logistic Regression vs MLP

1. Train / Validation / Test Split

The dataset was split into three parts: training, validation, and test sets. This was done so that the models could be trained and tested correctly. We used stratified splitting so that the class distribution (survival vs. death) stays the same in all subsets.

The goal of each dataset split is:

Training set—used to teach the models

Validation set: used to keep an eye on how well the model works while it's being built

Test set: only used once for the final evaluation

In [16]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop("DEATH_EVENT", axis=1)
y = df["DEATH_EVENT"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_temp, y_train, y_temp = train_test_split(
    X_scaled,
    y,
    test_size=0.3,
    stratify=y,
    random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.5,
    stratify=y_temp,
    random_state=42
)
```

2. Logistic Regression Model

Logistic regression is a linear classification model that predicts the probability of an outcome using a sigmoid function.

The mathematical form of logistic regression is:

$$y = \sigma(w^T x + b)$$

where:

- w represents model weights
- x represents input features
- b is a bias term
- σ is the sigmoid function

The expression $w^T x + b$ is a linear combination of the input features, which means the model produces a linear decision boundary.

Because of this linear structure, logistic regression cannot represent complex nonlinear relationships between variables.

In [17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

log_model = LogisticRegression()

log_model.fit(X_train, y_train)

val_pred = log_model.predict(X_val)

val_accuracy = accuracy_score(y_val, val_pred)

val_roc = roc_auc_score(
    y_val,
    log_model.predict_proba(X_val)[:,1]
)

print("Validation Accuracy:", val_accuracy)
print("Validation ROC-AUC:", val_roc)
```

```
Validation Accuracy: 0.8444444444444444
Validation ROC-AUC: 0.9177777777777778
```

3. Logistic Regression Test Performance

In [18]:
```
test_pred = log_model.predict(X_test)

test_accuracy = accuracy_score(y_test, test_pred)

test_roc = roc_auc_score(
    y_test,
    log_model.predict_proba(X_test)[:,1]
)

print("Test Accuracy:", test_accuracy)
print("Test ROC-AUC:", test_roc)
```

```
Test Accuracy: 0.8222222222222222
Test ROC-AUC: 0.8064516129032259
```

4. Multi-Layer Perceptron (MLP)

To go beyond what logistic regression can do, I built a simple feedforward neural network with one hidden layer. The idea was to see if a model with more flexibility could pick up on patterns that a straight-line decision boundary might miss.

The network follows a basic structure:

Input layer → Hidden layer (ReLU) → Output layer

The ReLU activation function is what gives the network its ability to learn nonlinear patterns. Without it, the model would just behave like another linear classifier, no matter how many layers we add. By introducing this nonlinearity, the MLP can capture more complex relationships between the clinical features and the target outcome.

5. Convert Data to PyTorch Tensors

In [19]:
```
import torch

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).view(-1,1)

X_val_tensor = torch.tensor(X_val, dtype=torch.float32)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32).view(-1,1)

X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).view(-1,1)
```

6. Define the MLP Model

In [20]:
```
import torch.nn as nn

class MLP(nn.Module):

    def __init__(self):

        super(MLP, self).__init__()

        self.network = nn.Sequential(
            nn.Linear(X_train.shape[1], 16),
            nn.ReLU(),
            nn.Linear(16,1),
            nn.Sigmoid()
        )

    def forward(self,x):

        return self.network(x)
```

7. Train the Neural Network

In [21]:
```
import torch.optim as optim

model = MLP()

criterion = nn.BCELoss()

optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 100
```

```
train_losses = []
val_losses = []

train_accs = []
val_accs = []
```

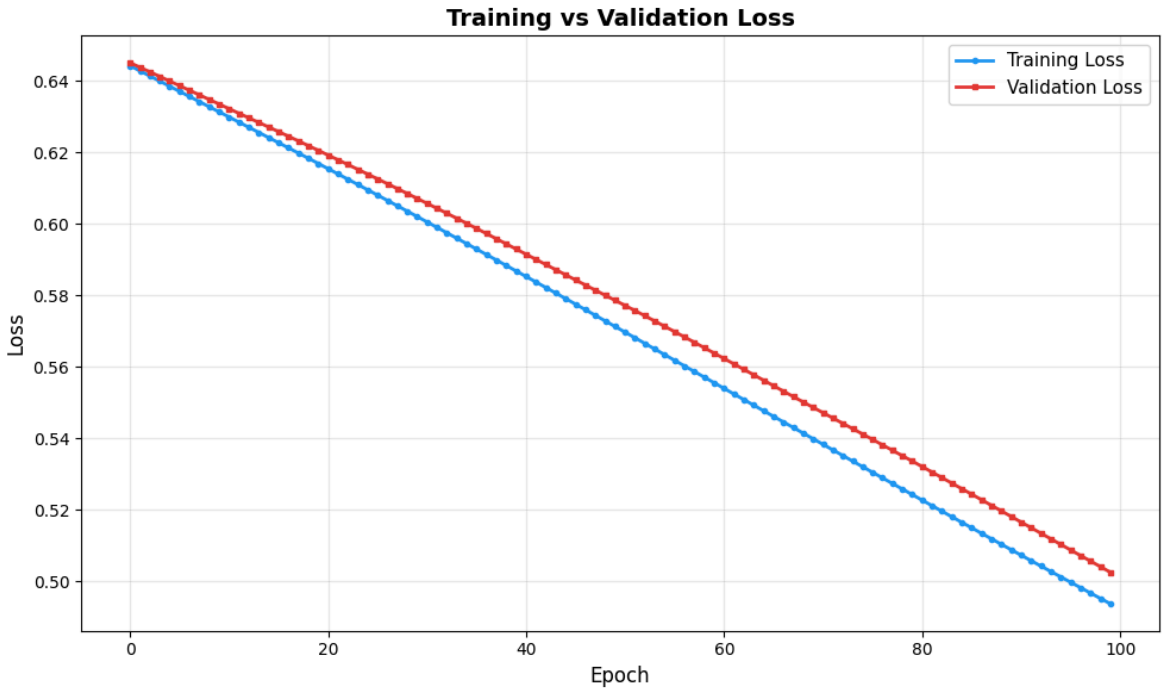
In [22]:
```
for epoch in range(epochs):

    model.train()

    optimizer.zero_grad()

    outputs = model(X_train_tensor)

    loss = criterion(outputs, y_train_tensor)

    loss.backward()

    optimizer.step()

    train_losses.append(loss.item())

    train_pred = (outputs.detach().numpy() > 0.5).astype(int)
    train_acc = accuracy_score(y_train, train_pred)

    train_accs.append(train_acc)

    model.eval()

    with torch.no_grad():

        val_outputs = model(X_val_tensor)

        val_loss = criterion(val_outputs, y_val_tensor)

        val_losses.append(val_loss.item())

        val_pred = (val_outputs.numpy() > 0.5).astype(int)

        val_acc = accuracy_score(y_val, val_pred)

        val_accs.append(val_acc)
```

8. Plot Learning Curves

Training vs Validation Loss

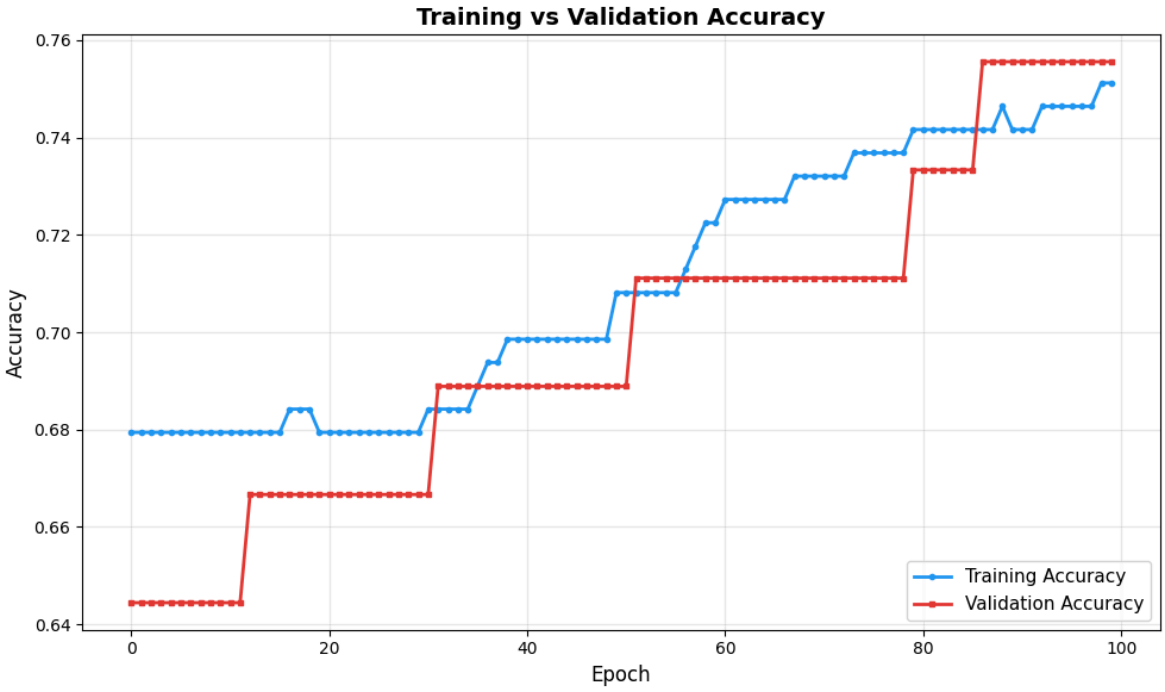
In [57]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.plot(train_losses, color='#2196F3', linewidth=2, label='Training Loss', mark
plt.plot(val_losses, color='#E53935', linewidth=2, label='Validation Loss', mark

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Training vs Validation Loss', fontsize=14, fontweight='bold')

plt.legend(fontsize=11, loc='upper right')
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Training vs Validation Accuracy

In [58]:
```
plt.figure(figsize=(10, 6))

plt.plot(train_accs, color='#2196F3', linewidth=2, label='Training Accuracy', ma
plt.plot(val_accs, color='#E53935', linewidth=2, label='Validation Accuracy', ma

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Training vs Validation Accuracy', fontsize=14, fontweight='bold')

plt.legend(fontsize=11, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



9. Evaluate MLP on Test Data

In [25]:
```
model.eval()

with torch.no_grad():

    test_outputs = model(X_test_tensor)

test_pred = (test_outputs.numpy() > 0.5).astype(int)

test_accuracy = accuracy_score(y_test, test_pred)

test_roc = roc_auc_score(y_test, test_outputs.numpy())

print("Test Accuracy:", test_accuracy)
print("Test ROC-AUC:", test_roc)
```

```
Test Accuracy: 0.6666666666666666
Test ROC-AUC: 0.6889400921658986
```

Required Analysis: Why Logistic Regression is a Linear Model

Logistic regression is called a linear model because, at its core, it makes predictions using a simple weighted sum of the input features.

The expression $w^T x + b$ takes each input variable, multiplies it by a learned weight, and adds them all together. This means the model draws a straight-line boundary to separate the two classes — it cannot bend or curve to fit more complex patterns in the data.

While the sigmoid function does convert this sum into a probability between 0 and 1, it does not change the fact that the relationship between the inputs and the prediction is still linear. In other words, the sigmoid just squashes the output — it does not add any ability to learn nonlinear patterns.

Comparison of Training Behaviour

Logistic regression tends to behave consistently during evaluation. Since it only has a small number of parameters to learn, there is less chance of it memorising the training data, which keeps overfitting under control.

The neural network, on the other hand, learns gradually by adjusting its weights over many training rounds (epochs). By plotting the training and validation accuracy at each epoch, we can see how the model's performance evolves over time.

One thing to watch out for is when the training accuracy keeps climbing but the validation accuracy levels off or starts to drop. This gap is a strong sign that the model is fitting too closely to the training data and not learning patterns that apply more broadly.

These learning curves are a useful diagnostic tool — they give us a clear visual picture of whether the model is actually generalising well or just memorising what it has already seen.

Representational Capacity vs Overfitting

By adding a hidden layer with the ReLU activation function, the neural network gains the ability to represent much more than a straight-line relationship. This extra flexibility means it can pick up on subtler, nonlinear connections between the clinical variables — something logistic regression simply cannot do.

The tricky part, though, is that our dataset is fairly small. When a powerful model is trained on limited data, there is a real risk that it starts learning the noise and quirks of the training set rather than genuine patterns.

This is where the validation metrics and learning curves become really important. They help us figure out whether the model is actually getting better at understanding the data, or just getting better at memorising it.

A good sign is when both training and validation performance improve together — that means the added complexity is genuinely helping. But if training accuracy keeps rising while validation accuracy falls behind, that widening gap is a clear warning that the model is overfitting.

Task 2 – Activation Function Comparison

1. Objective

The aim of this section is to explore how the choice of activation function affects the way a neural network learns. Activation functions are what give neural networks the ability to go beyond simple linear predictions — without them, stacking layers would not add any real benefit.

To keep the comparison fair, both models were set up with the exact same architecture and hyperparameters. The only thing that changes between them is the activation function used in the hidden layer.

The two activation functions compared are:

- **ReLU** (Rectified Linear Unit)
- **tanh** (Hyperbolic Tangent)

By watching how each model trains over time, we can get a better sense of how the activation function influences things like how quickly the model converges, how gradients flow through the network, and how stable the overall training process is.

2. Define Neural Network with Configurable Activation

To make it easy to compare how different activation functions perform, the neural network was set up so that the activation function can be swapped out without changing anything else in the model. This way, we can simply pass in the activation we want to test and keep everything else the same.

In [26]:
```
class MLPActivation(nn.Module):

    def __init__(self, activation):

        super(MLPActivation, self).__init__()

        self.model = nn.Sequential(
            nn.Linear(X_train.shape[1], 16),
            activation,
            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)
```

What it means

This network has:

a layer for input

a hidden layer with 16 neurones

an output layer for two types of classification

The activation function in the hidden layer will switch between ReLU and tanh.

3. Training Function

A reusable training function is created to train both models and record performance metrics.

In [28]:
```
def train_model(model):

    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
    epochs = 100

    train_losses = []
    val_losses = []

    train_acc = []
    val_acc = []

    for epoch in range(epochs):

        model.train()

        optimizer.zero_grad()

        outputs = model(X_train_tensor)

        loss = criterion(outputs, y_train_tensor)

        loss.backward()

        optimizer.step()

        train_losses.append(loss.item())

        train_pred = (outputs.detach().numpy() > 0.5).astype(int)
        train_acc.append(accuracy_score(y_train, train_pred))

        model.eval()

        with torch.no_grad():

            val_outputs = model(X_val_tensor)

            val_loss = criterion(val_outputs, y_val_tensor)

            val_losses.append(val_loss.item())

            val_pred = (val_outputs.numpy() > 0.5).astype(int)
            val_acc.append(accuracy_score(y_val, val_pred))

    return train_losses, val_losses, train_acc, val_acc
```

4. Train Model Using ReLU Activation

In [29]:
```
relu_model = MLPActivation(nn.ReLU())

relu_train_loss, relu_val_loss, relu_train_acc, relu_val_acc = train_model(relu_
```

Clarification

For negative inputs, ReLU outputs zero, and for positive inputs, it outputs the input value. This simple structure makes it easy for gradients to move around during training.

5. Train Model Using tanh Activation

In [30]:
```
tanh_model = MLPActivation(nn.Tanh())
```

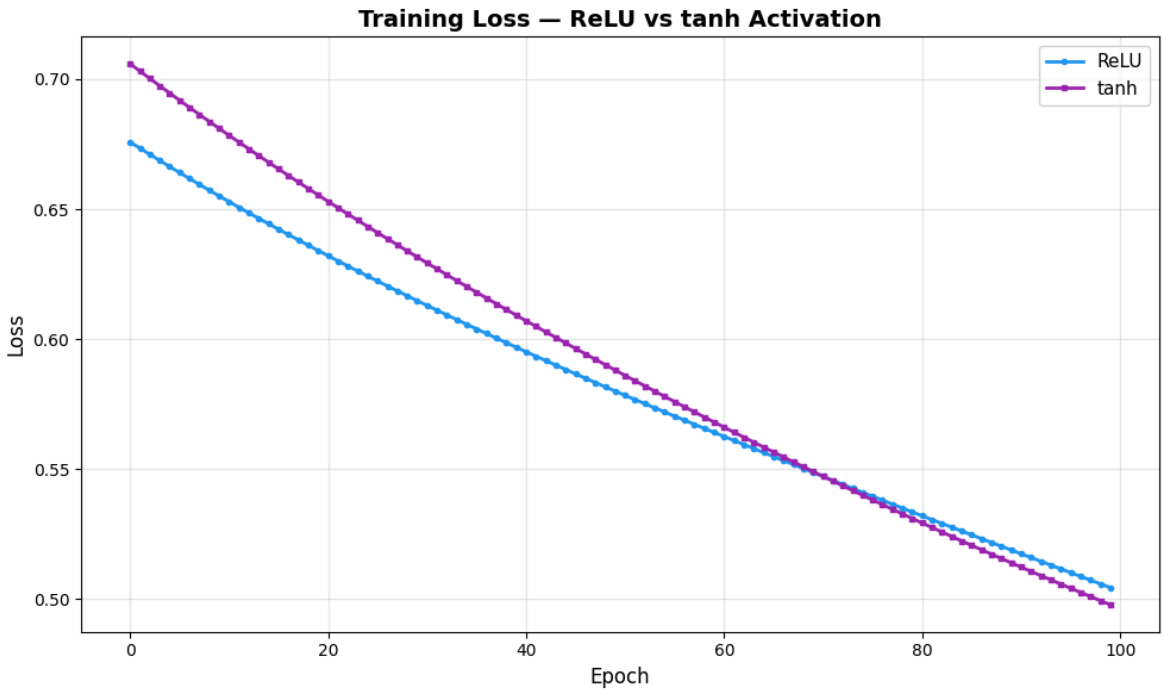
```
tanh_train_loss, tanh_val_loss, tanh_train_acc, tanh_val_acc = train_model(tanh_
```

Clarification

The tanh function takes values between −1 and 1 and changes the input in a smooth, nonlinear way.

6. Plot Training Loss

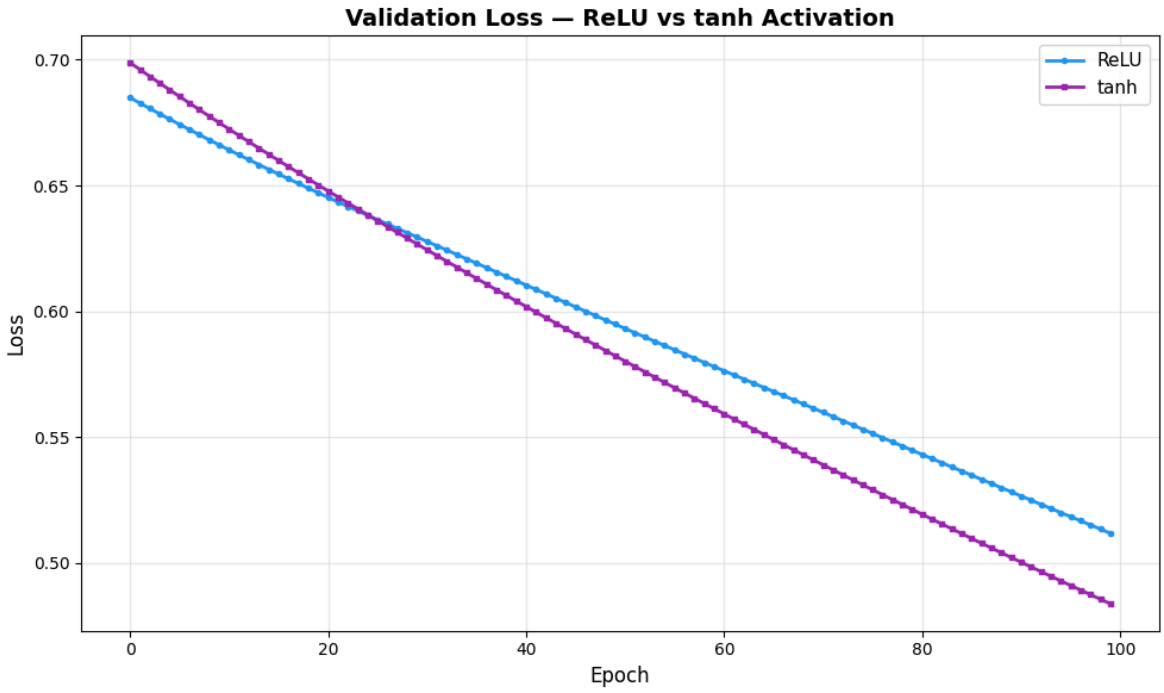
In [56]:
```
plt.figure(figsize=(10, 6))

plt.plot(relu_train_loss, color='#2196F3', linewidth=2, label='ReLU', marker='o'
plt.plot(tanh_train_loss, color='#9C27B0', linewidth=2, label='tanh', marker='s'

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Training Loss — ReLU vs tanh Activation', fontsize=14, fontweight='bo

plt.legend(fontsize=11, loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



7. Plot Validation Loss

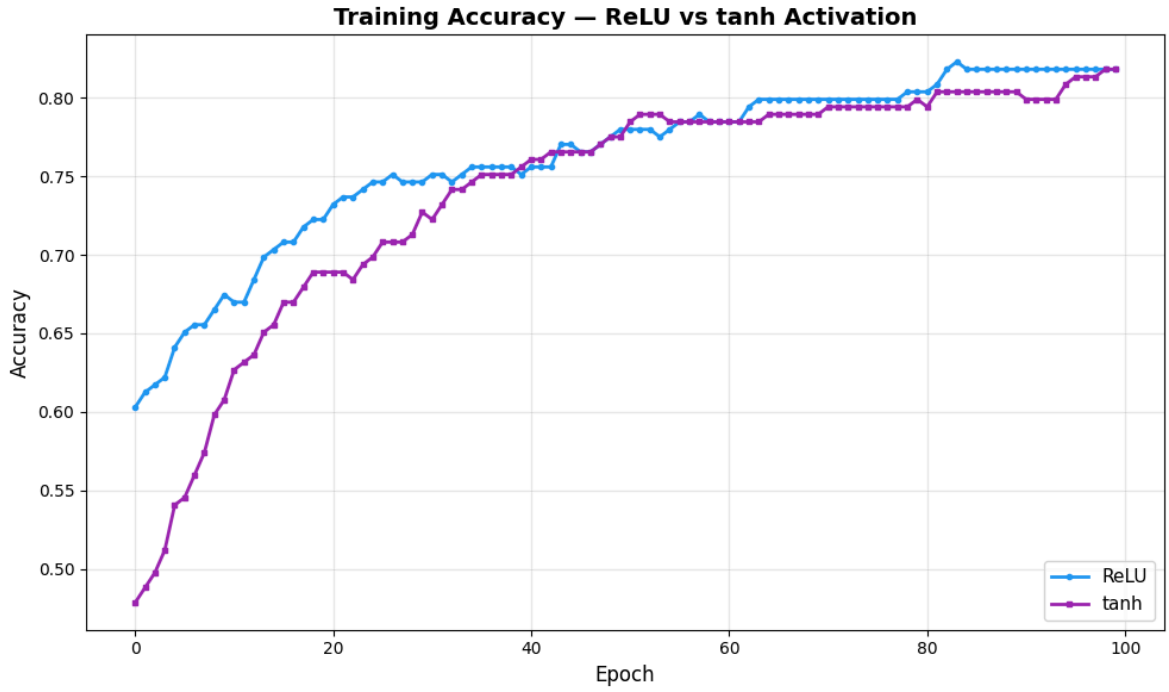
In [60]:
```
plt.figure(figsize=(10, 6))

plt.plot(relu_val_loss, color='#2196F3', linewidth=2, label='ReLU', marker='o',
plt.plot(tanh_val_loss, color='#9C27B0', linewidth=2, label='tanh', marker='s',

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Validation Loss — ReLU vs tanh Activation', fontsize=14, fontweight='

plt.legend(fontsize=11, loc='upper right')
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



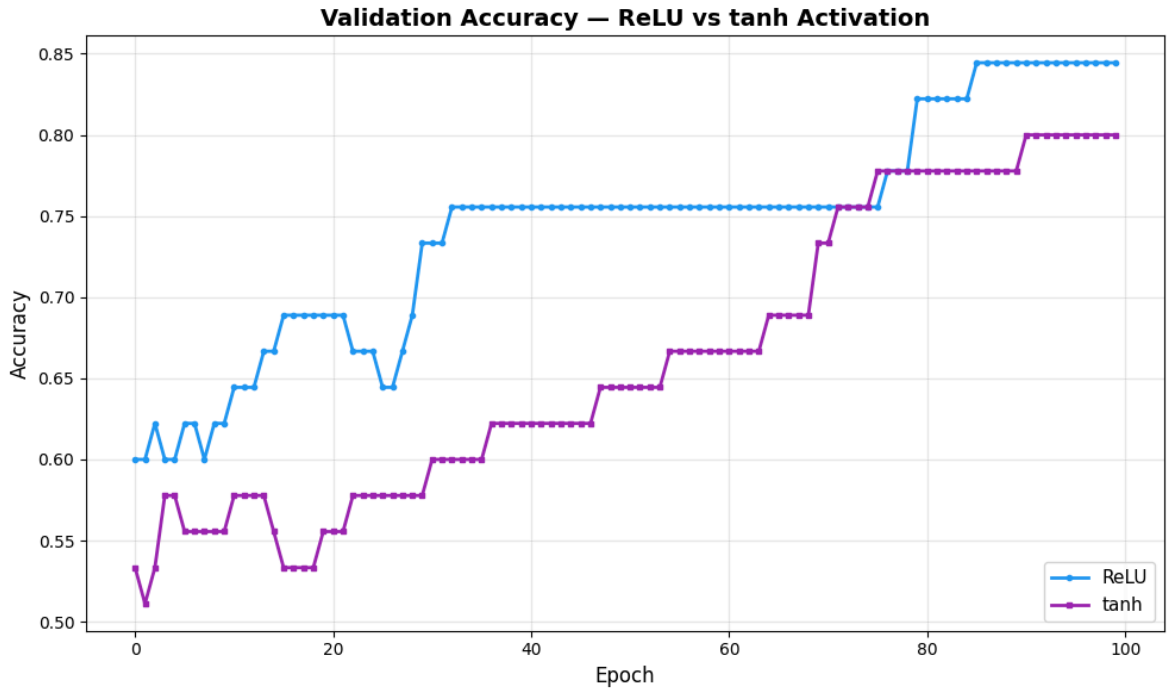
8. Plot Training Accuracy

In [61]:
```
plt.figure(figsize=(10, 6))

plt.plot(relu_train_acc, color='#2196F3', linewidth=2, label='ReLU', marker='o',
plt.plot(tanh_train_acc, color='#9C27B0', linewidth=2, label='tanh', marker='s',

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Training Accuracy — ReLU vs tanh Activation', fontsize=14, fontweight

plt.legend(fontsize=11, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



9. Plot Validation Accuracy

In [54]:
```
plt.figure(figsize=(10, 6))

plt.plot(relu_val_acc, color='#2196F3', linewidth=2, label='ReLU', marker='o', m
plt.plot(tanh_val_acc, color='#9C27B0', linewidth=2, label='tanh', marker='s', m

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Validation Accuracy — ReLU vs tanh Activation', fontsize=14, fontweig

plt.legend(fontsize=11, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Required Analysis: Speed of Convergence

The learning curves show that the ReLU activation model usually converges faster than the tanh model. ReLU keeps gradients from getting too big for positive inputs, which makes backpropagation work better.

Because of this, the ReLU model often gets stable training performance in fewer epochs.

Gradient Saturation in tanh

The tanh activation function squishes input values down to between −1 and 1. The function gets close to its limits when the input values get too big or too small.

Gradients get very small in these areas. This makes weight updates smaller during backpropagation, which slows down the learning process.

This is called gradient saturation.

The Dying ReLU Problem

ReLU can make training more efficient, but it can also cause the dying ReLU problem. When a neurone keeps getting negative inputs, its output becomes zero.

If this happens, the gradient for that neurone may also become zero, which means it won't be able to update during training.

Because of this, some neurones may no longer be able to help with learning.

Training Stability

The learning curves provide insight into training stability.

ReLU typically produces more stable and faster training due to its simple gradient behaviour. In contrast, tanh can lead to slower convergence and slightly less stable learning when gradients become saturated.

Despite this, tanh can sometimes provide smoother representations because its outputs are centred around zero.

Task 3: Overfitting and Capacity

1. Aim

This experiment examines the influence of model capacity on the performance of neural networks. Model capacity is the ability of a model to learn how different parts of the data are related to each other.

A network with more parameters can show more complicated patterns. But when the dataset is small, a model with a lot of capacity might learn noise in the training data instead of useful patterns. This is what happens when you overfit.

Two neural networks are compared to look into this behaviour:

A small network with 8 hidden units

Big network with 128 hidden units

Both networks use the same training method so that the size of the network can be seen clearly.

2. Define Neural Network Architecture

The neural network is defined so that the number of hidden units can be easily changed.

In [35]:
```
class CapacityMLP(nn.Module):

    def __init__(self, hidden_units):

        super(CapacityMLP, self).__init__()

        self.model = nn.Sequential(
            nn.Linear(X_train.shape[1], hidden_units),
            nn.ReLU(),
            nn.Linear(hidden_units, 1),
            nn.Sigmoid()
        )

    def forward(self, x):

        return self.model(x)
```

What it means

The structure of the network stays the same, but the number of hidden neurones changes to change the model's capacity.

3. Training Function

A reusable training function is used to train both networks and record performance across epochs.

In [36]:
```
def train_capacity_model(model):

    criterion = nn.BCELoss()

    optimizer = optim.Adam(model.parameters(), lr=0.001)

    epochs = 100

    train_losses = []
    val_losses = []

    train_acc = []
    val_acc = []

    for epoch in range(epochs):
```

```
        model.train()

        optimizer.zero_grad()

        outputs = model(X_train_tensor)

        loss = criterion(outputs, y_train_tensor)

        loss.backward()

        optimizer.step()

        train_losses.append(loss.item())

        train_pred = (outputs.detach().numpy() > 0.5).astype(int)

        train_acc.append(accuracy_score(y_train, train_pred))

        model.eval()

        with torch.no_grad():

            val_outputs = model(X_val_tensor)

            val_loss = criterion(val_outputs, y_val_tensor)

            val_losses.append(val_loss.item())

            val_pred = (val_outputs.numpy() > 0.5).astype(int)

            val_acc.append(accuracy_score(y_val, val_pred))

    return train_losses, val_losses, train_acc, val_acc
```

4. Train Small Network (8 Hidden Units)

In [37]:
```
small_model = CapacityMLP(hidden_units=8)

small_train_loss, small_val_loss, small_train_acc, small_val_acc = train_capacit
```

Clarification

This model doesn't have a lot of capacity, which means it has fewer parameters and is less likely to fit the training data too closely.

5. Train Large Network (128 Hidden Units)

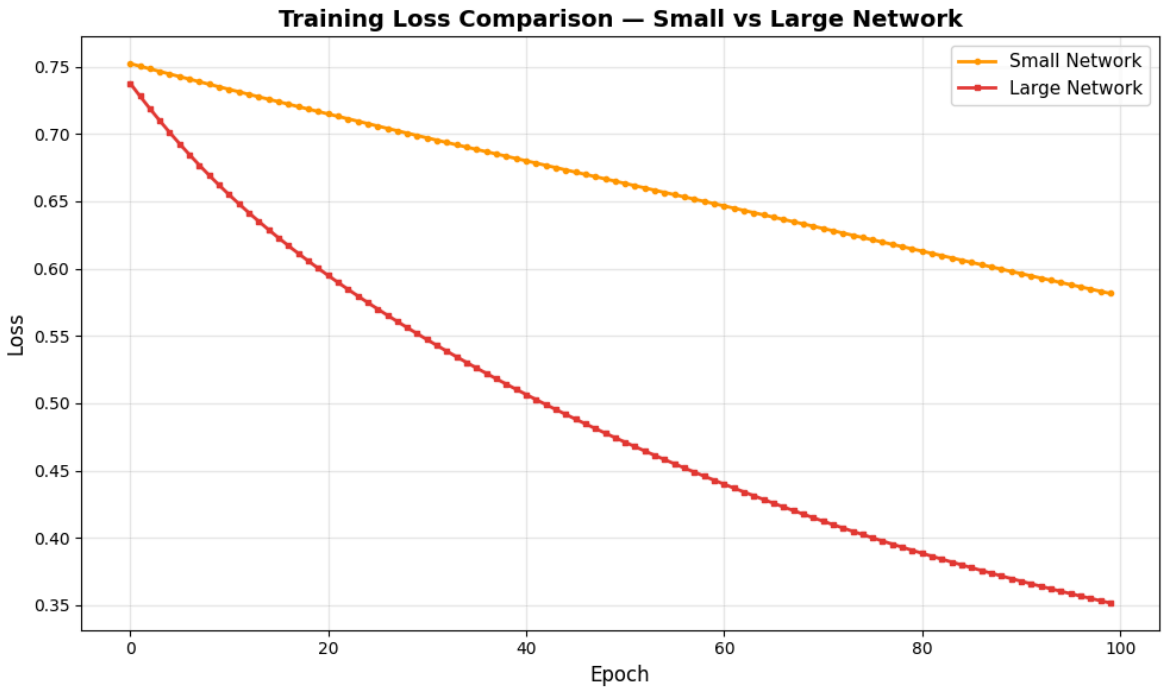
In [38]:
```
large_model = CapacityMLP(hidden_units=128)

large_train_loss, large_val_loss, large_train_acc, large_val_acc = train_capacit
```

Clarification

This model can learn more complicated relationships because it has more capacity. But this could cause overfitting if the dataset is small.

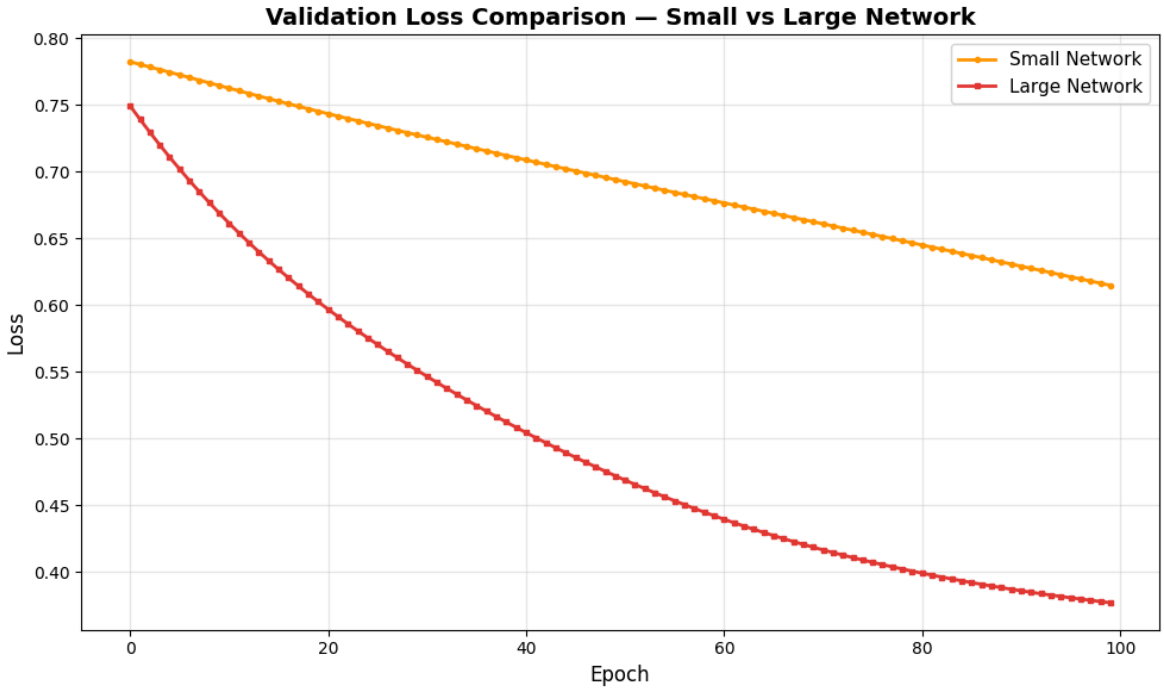
6. Plot Training Loss

In [53]:
```
plt.figure(figsize=(10, 6))

plt.plot(small_train_loss, color='#FF9800', linewidth=2, label='Small Network',
plt.plot(large_train_loss, color='#E53935', linewidth=2, label='Large Network',

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Training Loss Comparison — Small vs Large Network', fontsize=14, font

plt.legend(fontsize=11, loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



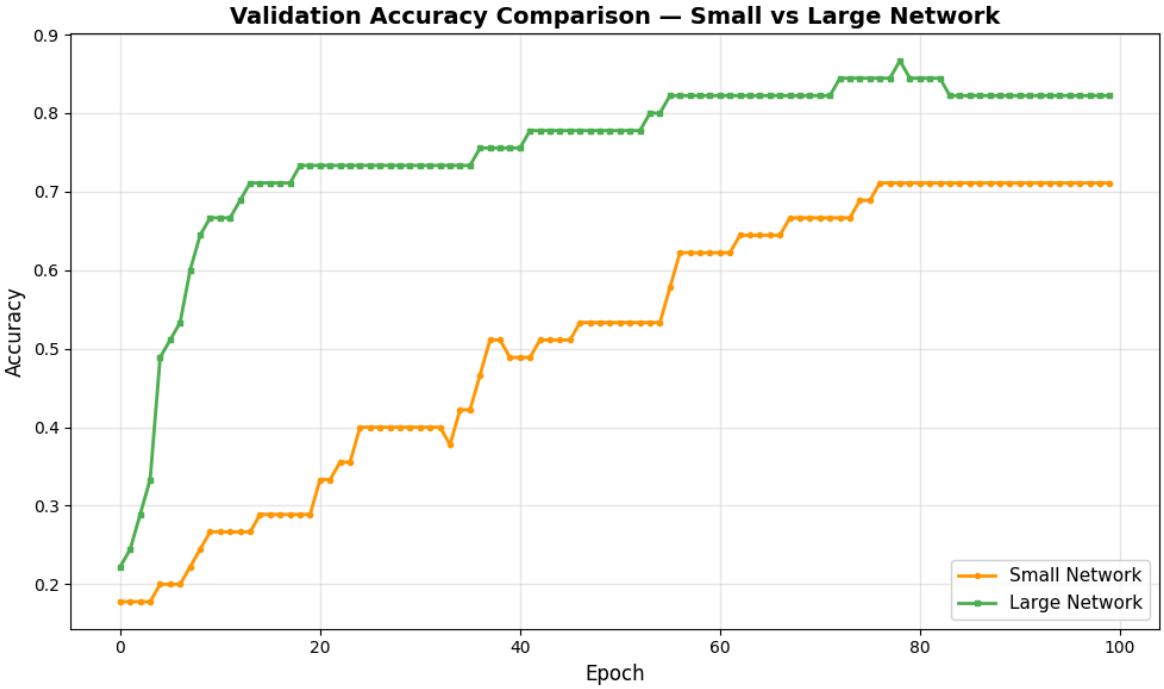
7. Plot Validation Loss

In [52]:
```
plt.figure(figsize=(10, 6))

plt.plot(small_val_loss, color='#FF9800', linewidth=2, label='Small Network', ma
plt.plot(large_val_loss, color='#E53935', linewidth=2, label='Large Network', ma

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Validation Loss Comparison — Small vs Large Network', fontsize=14, fo

plt.legend(fontsize=11, loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



8. Plot Validation Accuracy

In [51]:
```
plt.figure(figsize=(10, 6))

plt.plot(small_val_acc, color='#FF9800', linewidth=2, label='Small Network', mar
plt.plot(large_val_acc, color='#4CAF50', linewidth=2, label='Large Network', mar

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Validation Accuracy Comparison — Small vs Large Network', fontsize=14

plt.legend(fontsize=11, loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



9. Use Regularisation (L2 Weight Decay)

L2 regularisation is used on the big network to stop it from overfitting.

L2 regularisation punishes large weights during training, which helps the model learn patterns that are easier to understand and apply to other situations.

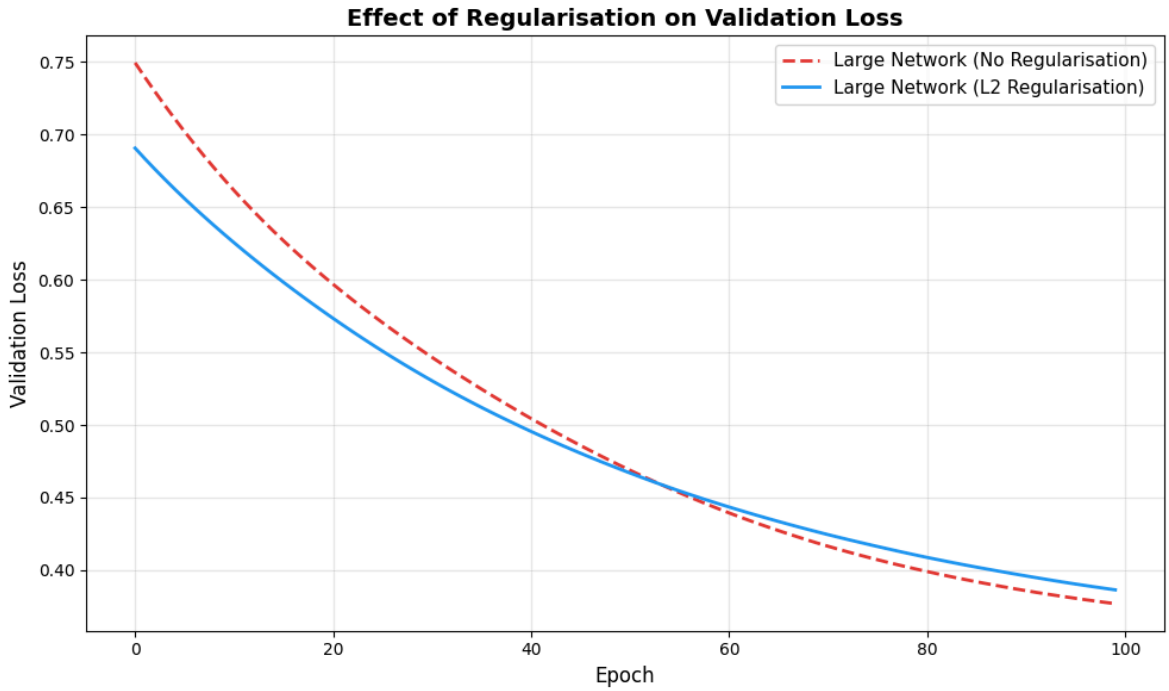
In [42]:
```
def train_regularised_model(model):

    criterion = nn.BCELoss()

    optimizer = optim.Adam(
        model.parameters(),
        lr=0.001,
        weight_decay=0.01
    )

    epochs = 100

    train_losses = []
    val_losses = []

    for epoch in range(epochs):

        model.train()

        optimizer.zero_grad()

        outputs = model(X_train_tensor)

        loss = criterion(outputs, y_train_tensor)

        loss.backward()

        optimizer.step()

        train_losses.append(loss.item())

        model.eval()

        with torch.no_grad():

            val_outputs = model(X_val_tensor)

            val_loss = criterion(val_outputs, y_val_tensor)

            val_losses.append(val_loss.item())

    return train_losses, val_losses
```

10. Train Regularised Large Network

In [43]:
```
regularised_model = CapacityMLP(hidden_units=128)

reg_train_loss, reg_val_loss = train_regularised_model(regularised_model)
```

11. Plot Regularisation Effect

In [50]:
```
plt.figure(figsize=(10, 6))

plt.plot(large_val_loss, color='#E53935', linewidth=2, label='Large Network (No
plt.plot(reg_val_loss, color='#2196F3', linewidth=2, label='Large Network (L2 Re

plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Validation Loss', fontsize=12)
plt.title('Effect of Regularisation on Validation Loss', fontsize=14, fontweight

plt.legend(fontsize=11, loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Analysis Needed: Impact of Model Capacity

The small network with fewer hidden units can't represent as much information. It may not be able to find all the complicated patterns in the data, but it is more likely to generalise because it has fewer parameters.

On the other hand, the bigger network with 128 hidden units can represent things much better. This helps it fit the training data better, but it also makes it more likely to overfit, especially when the dataset is small.

You can see this behaviour by looking at the training and validation curves side by side.

Proof of Overfitting

Overfitting happens when the model learns patterns that only apply to the training data instead of patterns that apply to new data.

Signs that a model is overfitting include:

very high accuracy in training

lower accuracy of validation

growing difference between performance on training and validation tests

The bigger network may show these traits because it is more complicated.

Effect of Regularisation

During training, L2 regularisation adds a penalty for big weight values. This prevents the model from relying excessively on specific features and encourages smoother decision boundaries.

The model is less likely to overfit when regularisation is applied to the large network, and it may also have more stable validation performance.

Task 4—Responsible Review

1. Goal

The last step is to test the chosen models on the test dataset after choosing the final model architectures in the previous tasks.

The test set is meant to give an unbiased guess about how well the model will work on new data. The test dataset can only be used once, after all the decisions about how to design the model have been made. This is different from the training and validation datasets.

This makes sure that the results reported are what the model would do in real life.

2. Final Model Evaluation on the Test Set

Two performance metrics are used to evaluate the chosen model:

Correctness

ROC-AUC

These numbers help figure out how well the model predicts how long patients will live.

In [45]:
```
model.eval()

with torch.no_grad():

    test_outputs = model(X_test_tensor)

test_predictions = (test_outputs.numpy() > 0.5).astype(int)

test_accuracy = accuracy_score(y_test, test_predictions)

test_roc = roc_auc_score(y_test, test_outputs.numpy())

print("Test Accuracy:", test_accuracy)
print("Test ROC-AUC:", test_roc)
```

```
Test Accuracy: 0.6666666666666666
Test ROC-AUC: 0.6889400921658986
```

3. Matrix of Confusion

A confusion matrix gives a full picture of how well predictions worked out.

It shows:

True Positives

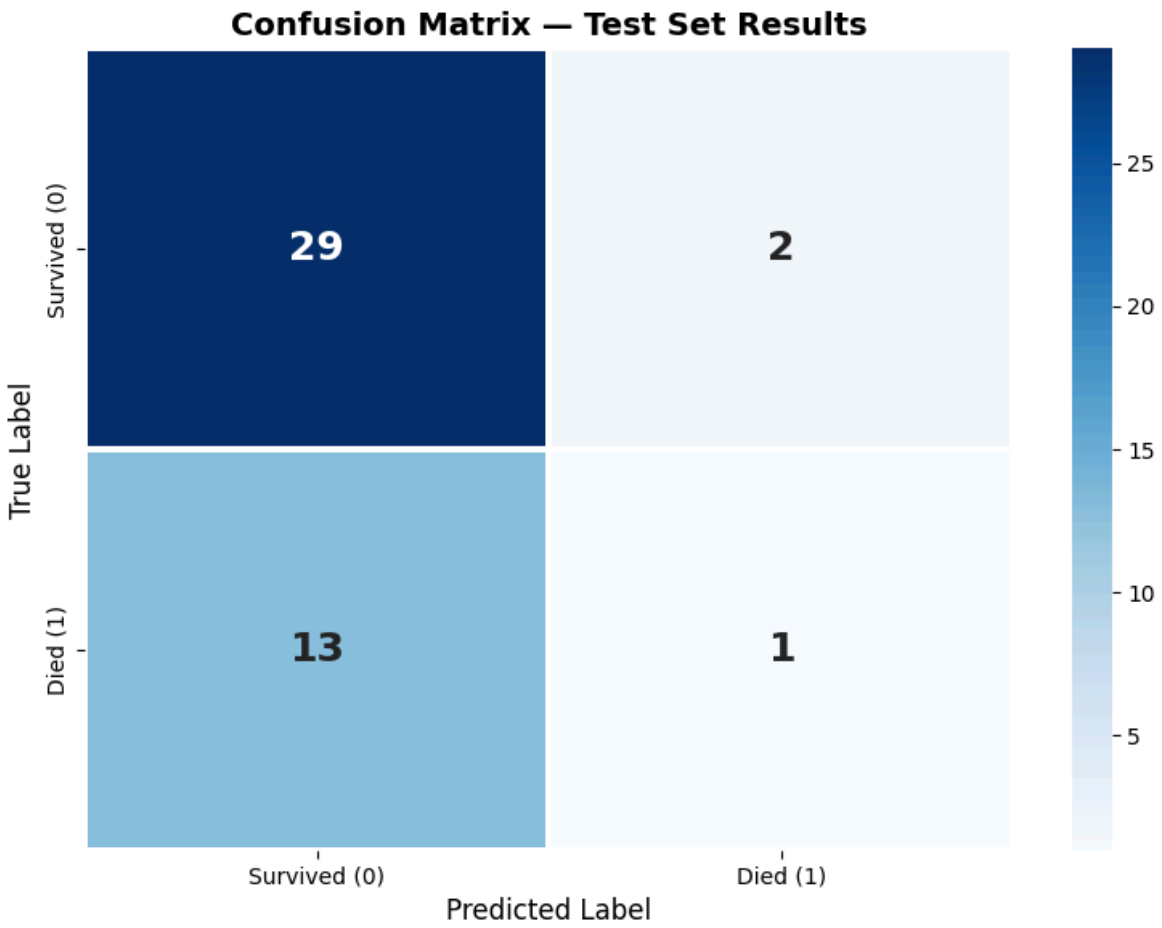
Real Negatives

False Positives

False Negatives

This helps understand how the model performs on each class.

In [49]:
```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, test_predictions)

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            annot_kws={"size": 18, "fontweight": "bold"},
            linewidths=1.5, linecolor='white',
            xticklabels=['Survived (0)', 'Died (1)'],
            yticklabels=['Survived (0)', 'Died (1)'])

plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)
plt.title('Confusion Matrix — Test Set Results', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()
```



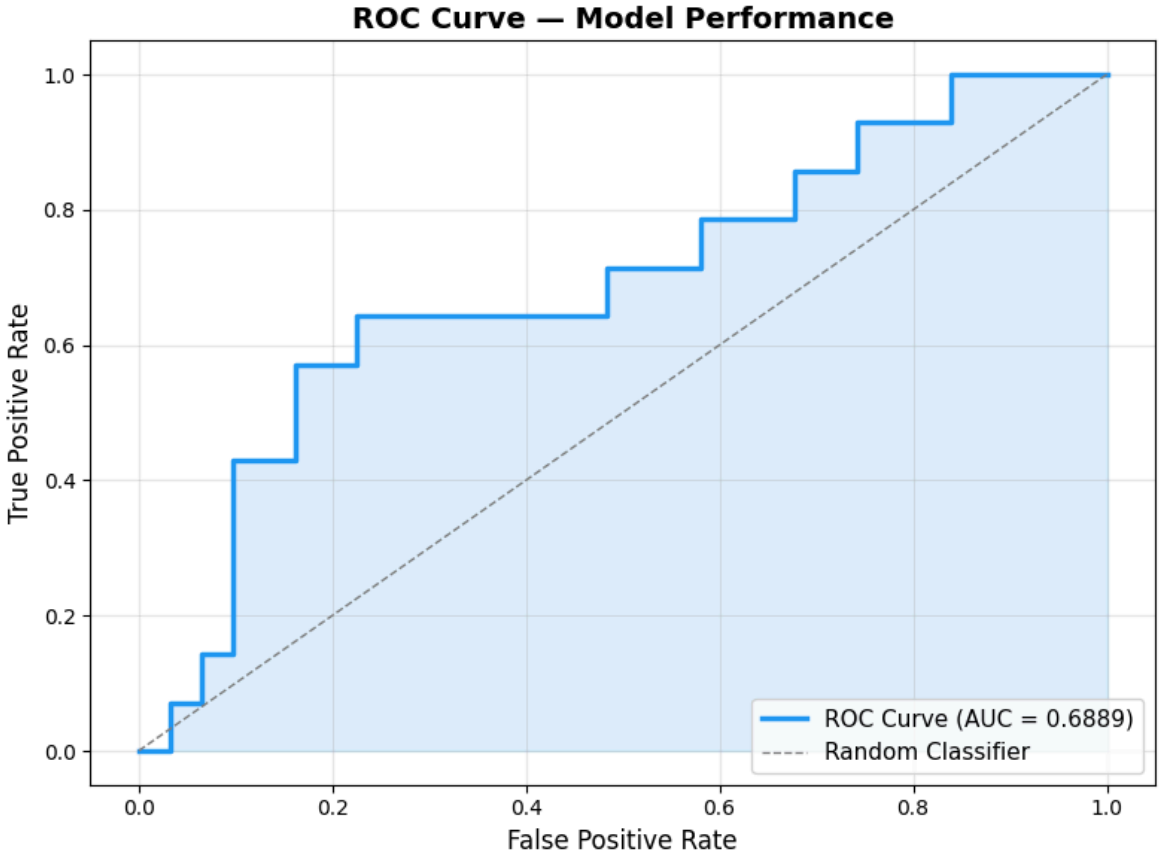
4. ROC Curve

The ROC curve shows how well the model separates the two classes at different levels of classification.

A model that can tell the difference between things better will make a curve that is closer to the top left corner of the graph.

In [48]:
```
from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve(y_test, test_outputs.numpy())
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))

# Plot ROC curve
plt.plot(fpr, tpr, color='#2196F3', linewidth=2.5, label=f'ROC Curve (AUC = {roc

# Plot diagonal reference line
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', linewidth=1, label='Randc

# Fill area under curve
plt.fill_between(fpr, tpr, alpha=0.15, color='#2196F3')

plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curve — Model Performance', fontsize=14, fontweight='bold')

plt.legend(loc='lower right', fontsize=11)
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Required Analysis

Role of Training Data

The model parameters are learned from the training dataset. The neural network changes its weights during training to lower the loss function and make predictions more accurate.

Because the model sees the training data over and over again during optimisation, the performance on the training data often looks better than the performance on data it hasn't seen before.

So, training metrics by themselves can't give you a good idea of how well something will generalise.

Role of Validation Data

The validation dataset is used during model development to evaluate different architectures and hyperparameters.

For example, in previous tasks the validation dataset was used to:

compare logistic regression and neural networks

compare activation functions

examine the impact of network size

evaluate regularisation techniques

The validation set helps identify which model configuration performs best before performing the final test evaluation.

Why the Test Set Must Not Guide Architectural Decisions

The test dataset should only be used after all model decisions have been finalised. If the test data is used repeatedly during model development, the model may indirectly adapt to the test data.

This leads to data leakage, where the evaluation results become overly optimistic and no longer reflect true generalisation performance.

Using the test set only once ensures that the reported metrics represent an unbiased estimate of how the model will perform on new unseen data.

Why Small Numerical Differences Should Not Be Overinterpreted

When working with small datasets, performance differences between models may appear due to random variation in the training process or data sampling.

For example, a difference of 1–2% in accuracy does not necessarily indicate that one model is significantly better than another.

Therefore, it is important to interpret small numerical differences cautiously and focus on broader trends such as:

learning behaviour

stability of training curves

consistency between training and validation performance

This approach leads to more reliable conclusions about model performance.

Final Summary

In this task, the selected model was evaluated using a separate test dataset to obtain an unbiased estimate of performance.

By clearly separating training, validation, and test datasets, the evaluation process ensures that the results reflect the model's ability to generalise to new data rather than memorising the training examples.

Responsible evaluation practices are essential for building reliable machine learning models, particularly when working with limited datasets.